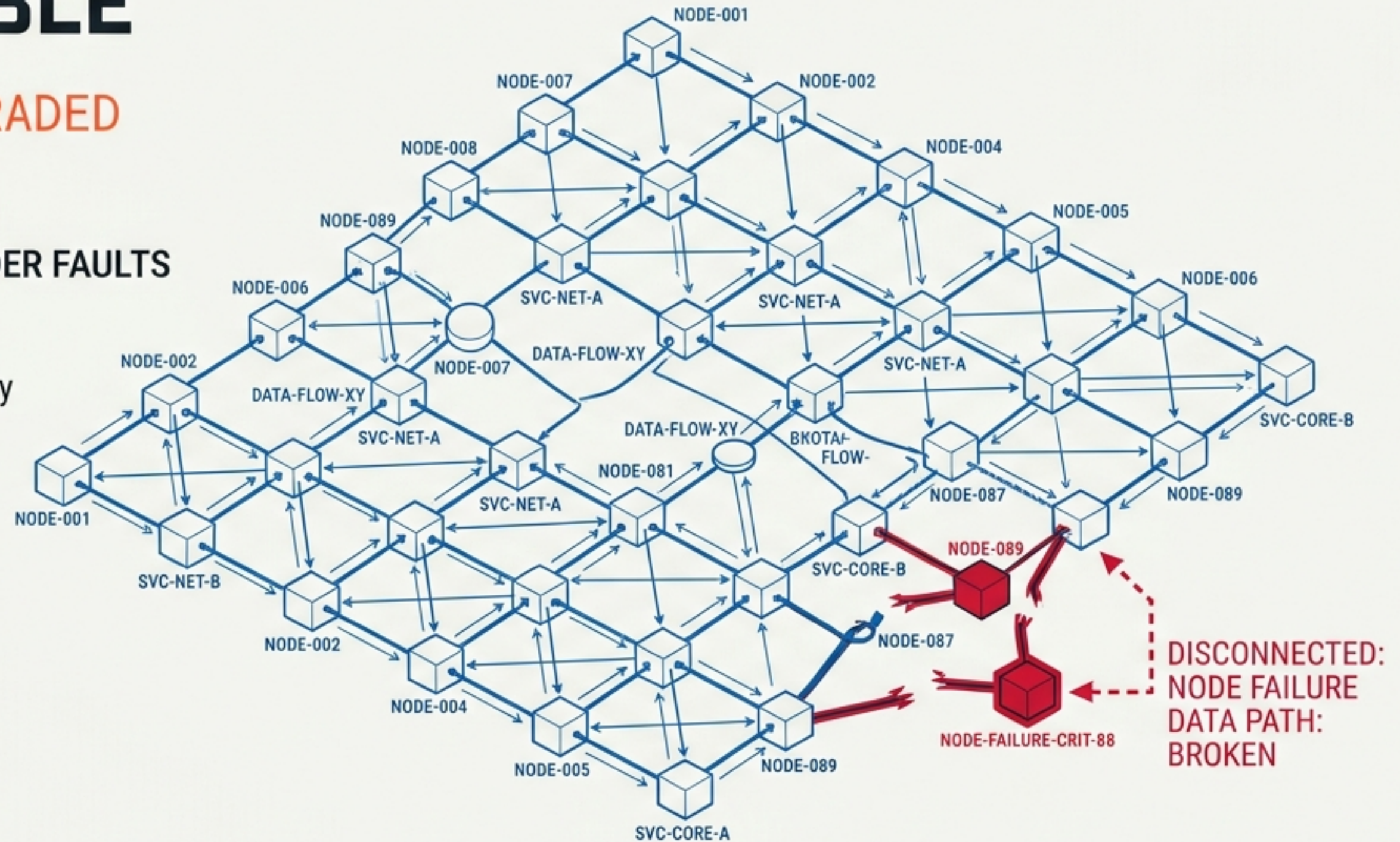


ARCHITECTURE FOR THE UNPREDICTABLE

SYSTEM STATUS: DEGRADED

FOCUS: AVAILABILITY & SERVICES
OBJECTIVE: RUNTIME UPTIME UNDER FAULTS

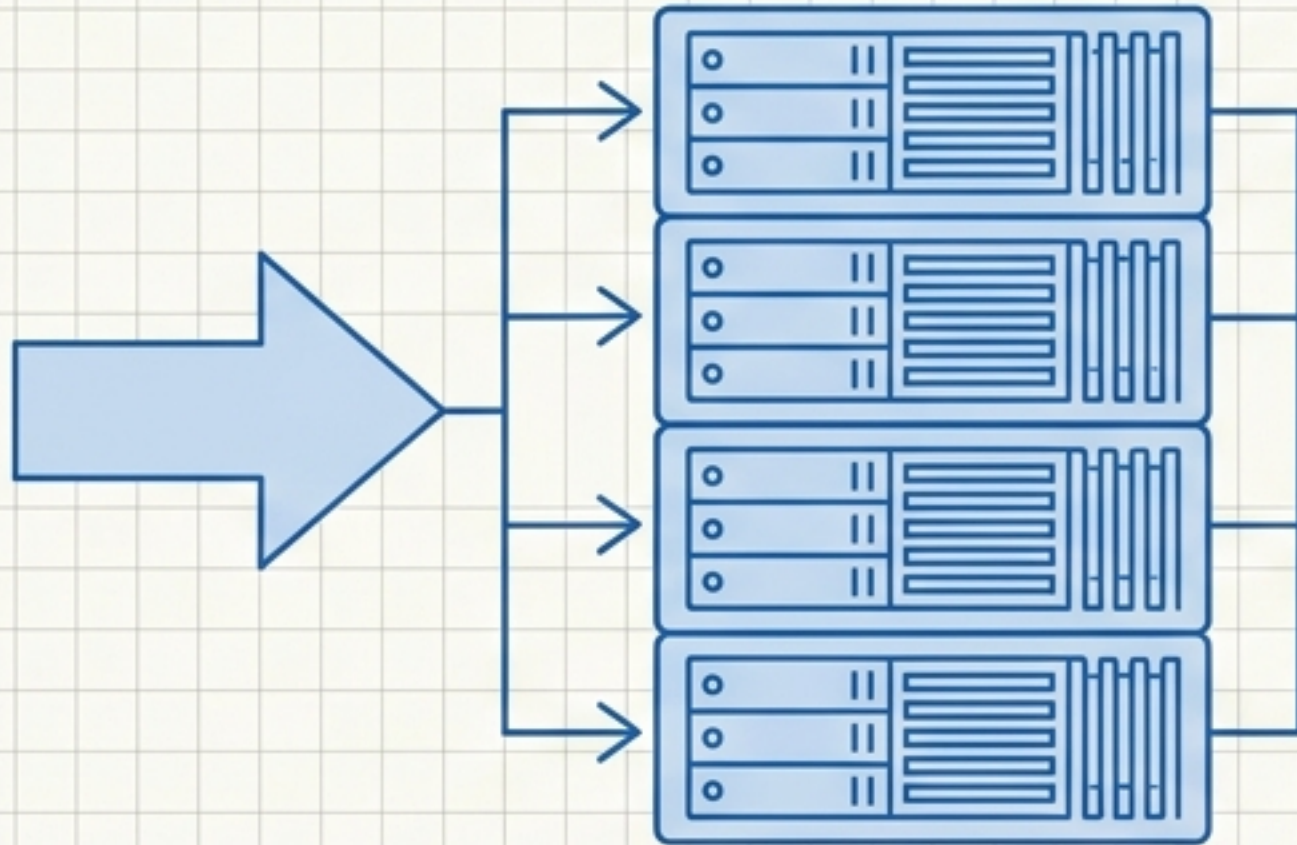
Availability is the fraction of time a system correctly serves users, especially when relying on services operated by others.



A system can be blazingly fast yet completely fragile

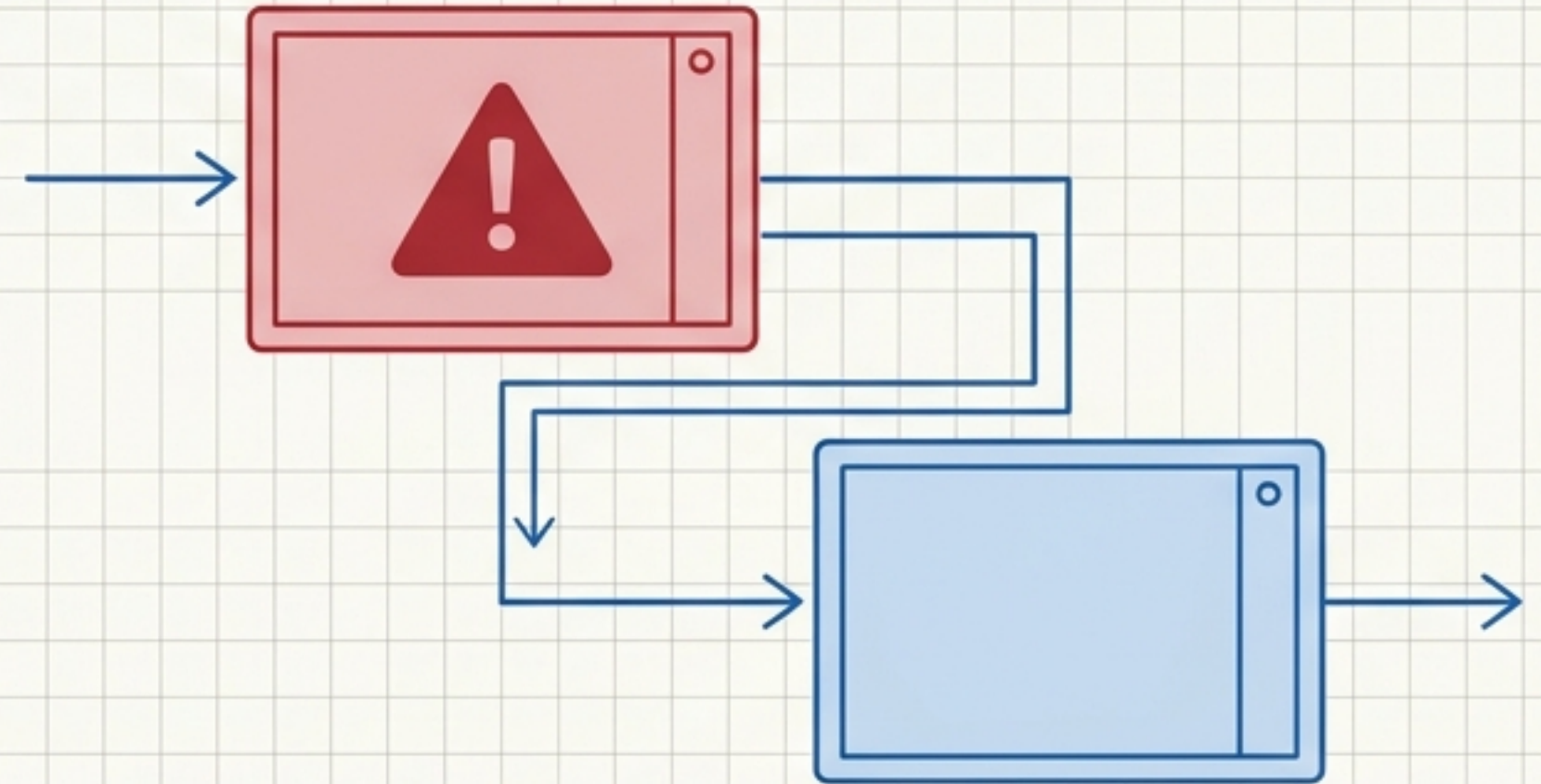
Scalability

Handling workload growth
without hitting saturation.



Availability

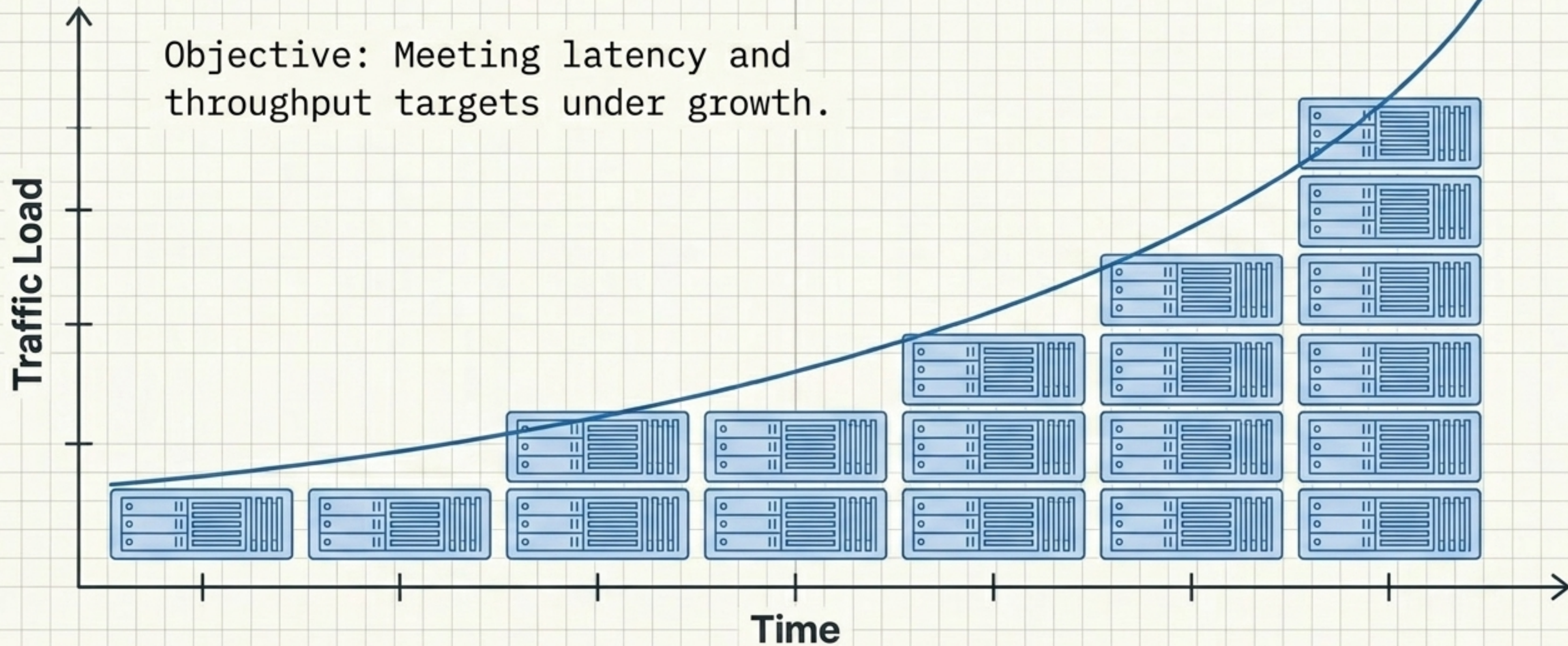
Surviving faults when parts
inevitably disappear.



Scalability is about Capacity

Workload versus Resources

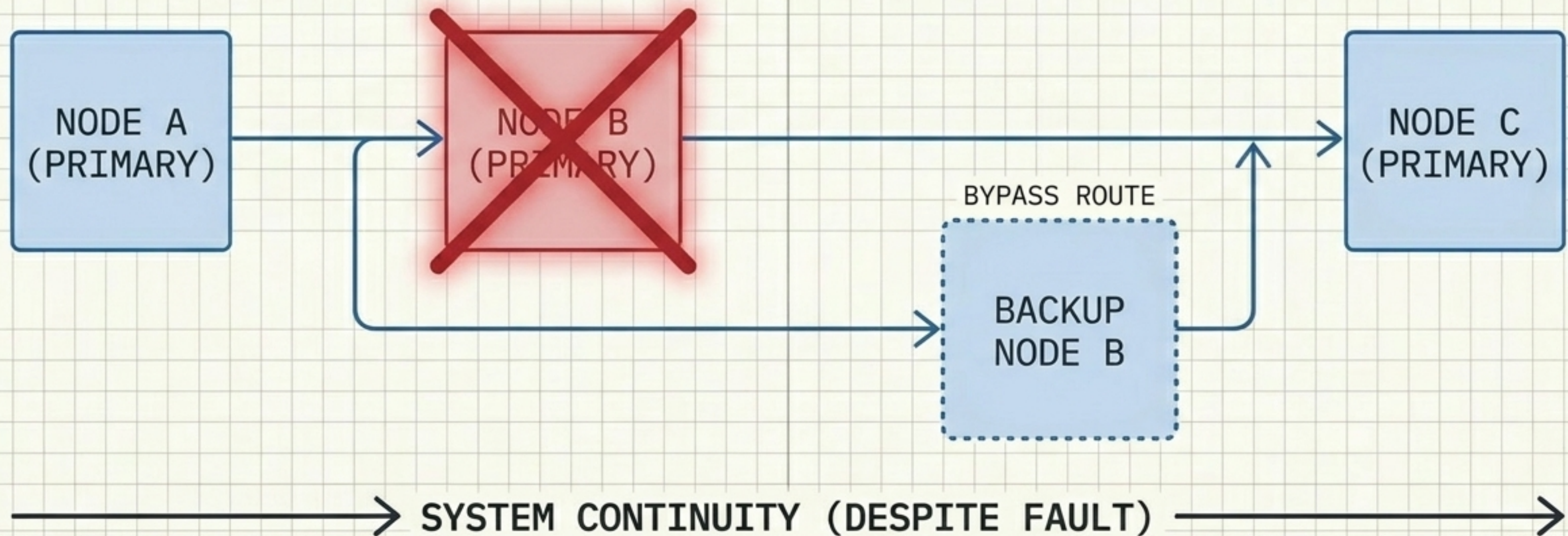
Objective: Meeting latency and throughput targets under growth.



Availability is about Faults

Uptime versus Failure

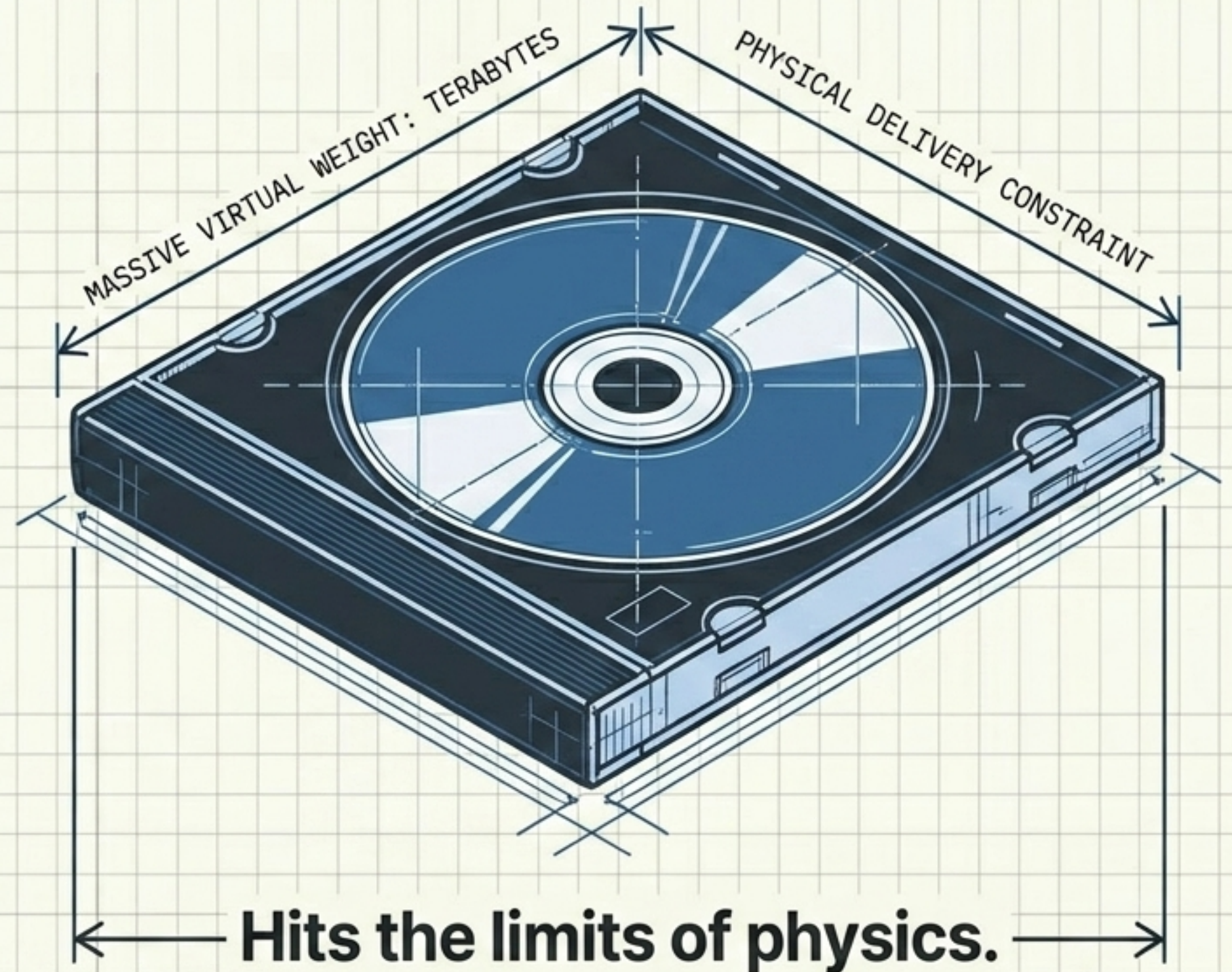
Objective: The system correctly serves users even when internal parts break.



The Map Dilemma: Component Model

`getMap(lat, lng, zoom)`

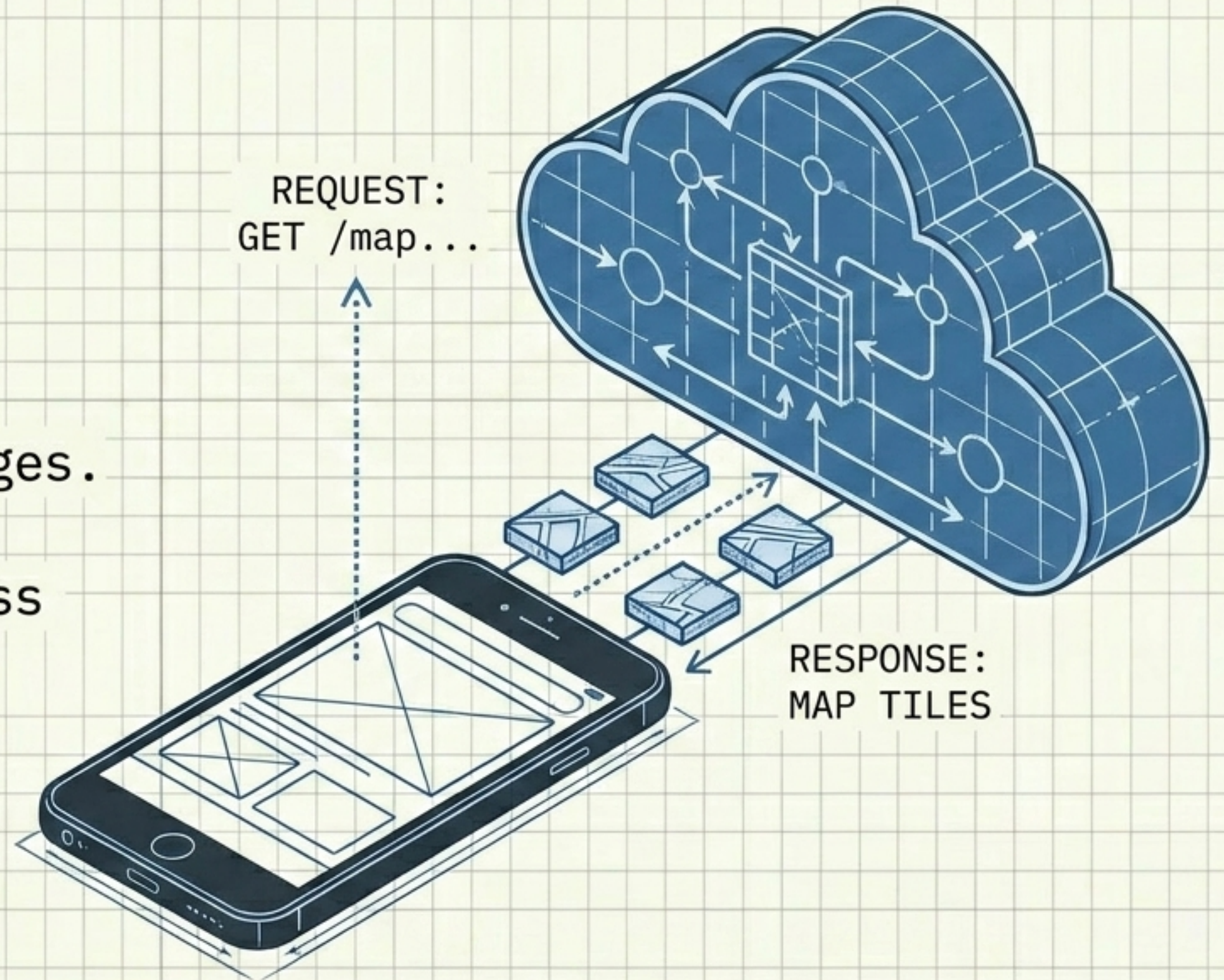
- Self-contained. Requires shipping terabytes of planet data.



The Map Dilemma: Service Model

`GET /map?lat=x&lng=y`

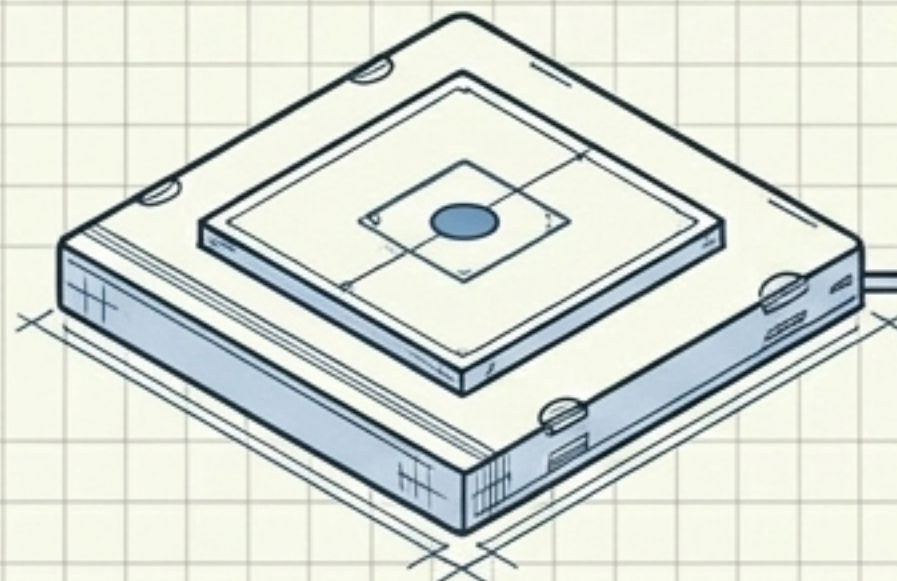
- Remote API. Thin client sending lightweight messages.
- Enables real-time freshness and partial delivery.



Service = Software Someone Else Runs

WARNING: Services move blame lines. Your application's **uptime depends entirely on external SLAs.**

Your Environment

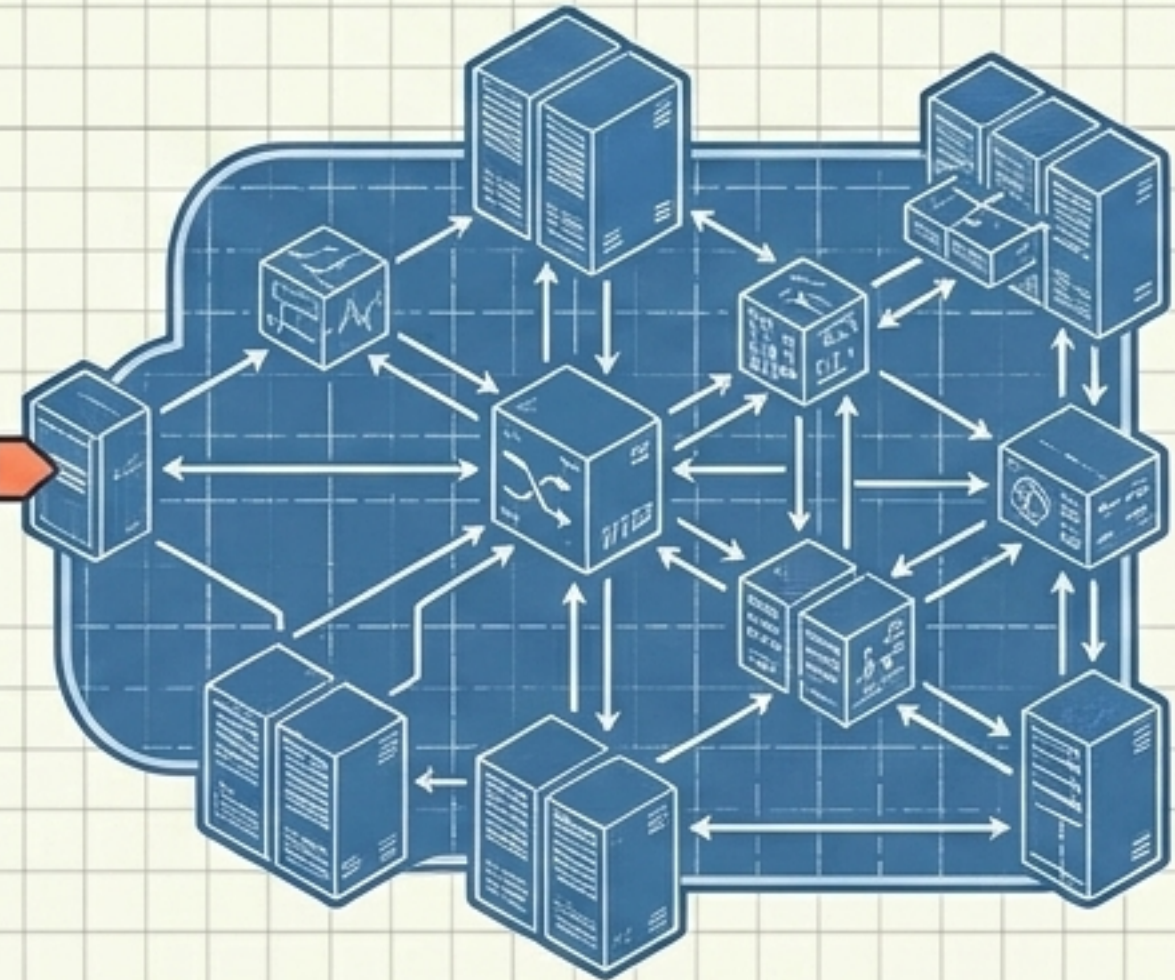


Your Application

Fragile
Connector



Provider Cloud



Shifting Operational Ownership

	Component (You Run)	Service (They Run)
Ownership	Outage is your queue. Internal help desk resolves issues.	Provider controls deploys. Contractual vendor coupling.
Packaging	Language/Framework specific. Standardized by Docker.	Published URL. Language-agnostic.

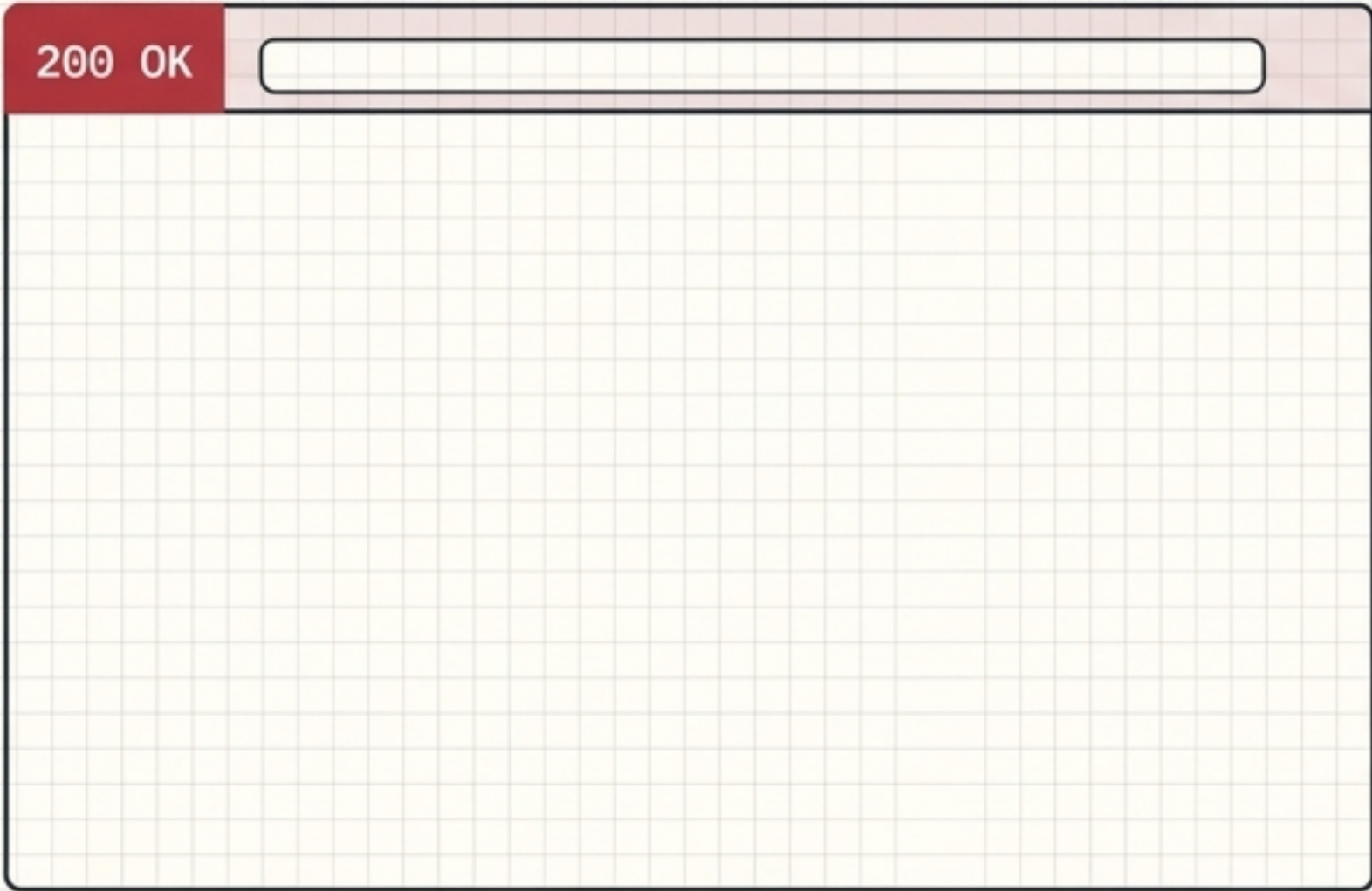
Shifting Integration and Economics

	Component (You Run)	Service (They Run)
Integration	Shared memory. In-process communication.	Remote connectors. HTTP/JSON over the public internet.
Business Model	Per-deploy / install. Shrink-wrap economics.	Per-use / subscription. Continuous operation.

Reachable is not the same thing as Useful

Reachable

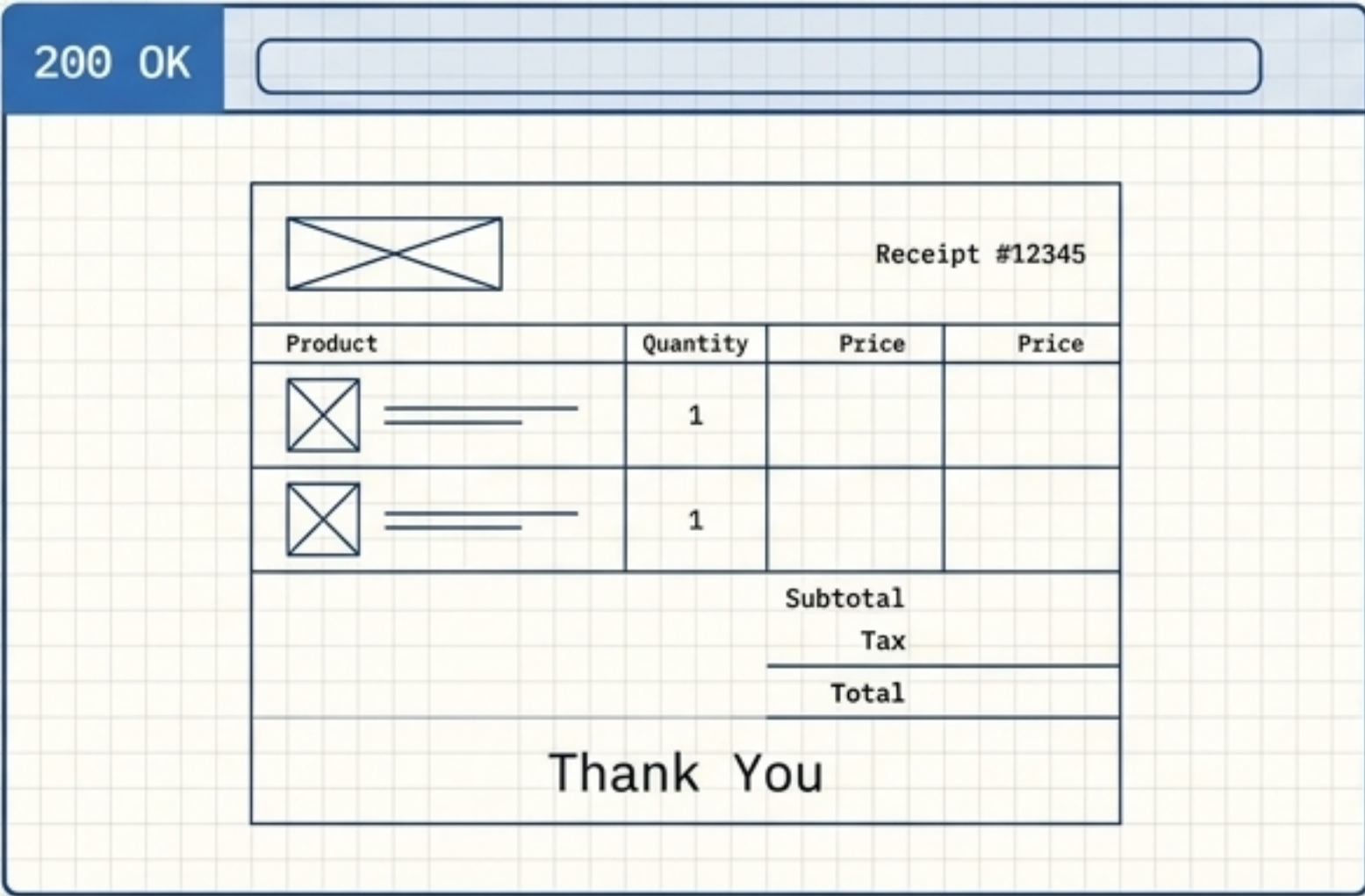
Process is up. Host answers an HTTP ping.



System reachable, but useless content.

Useful

Correct business outcome achieved within the expected timeframe.



System performing useful work.

The Math of Reliability

MTBF: Mean Time Between Failures

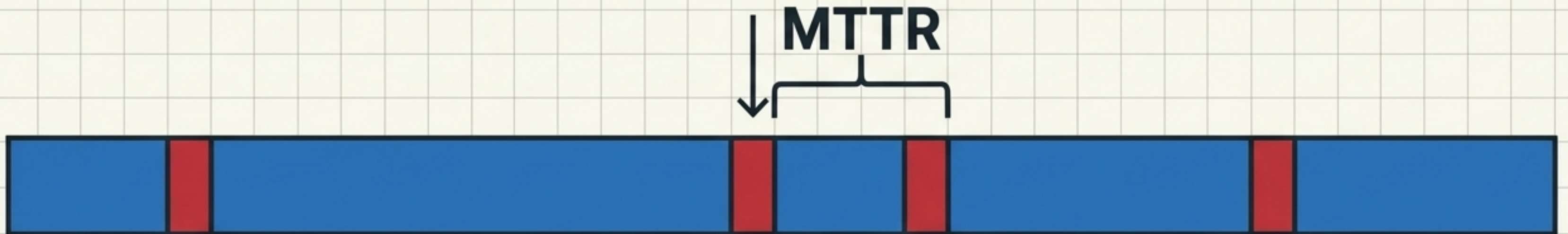
Reliability of components. How rarely you break.



The Math of Recovery

MTTR: Mean Time To Restore

Ops skill, automation, and failover.
How fast you recover.



The Dependency Chain Trap

Your SLO is constrained by your dependencies.
Model the chain, not just one tier.



Multiplicative Fragility

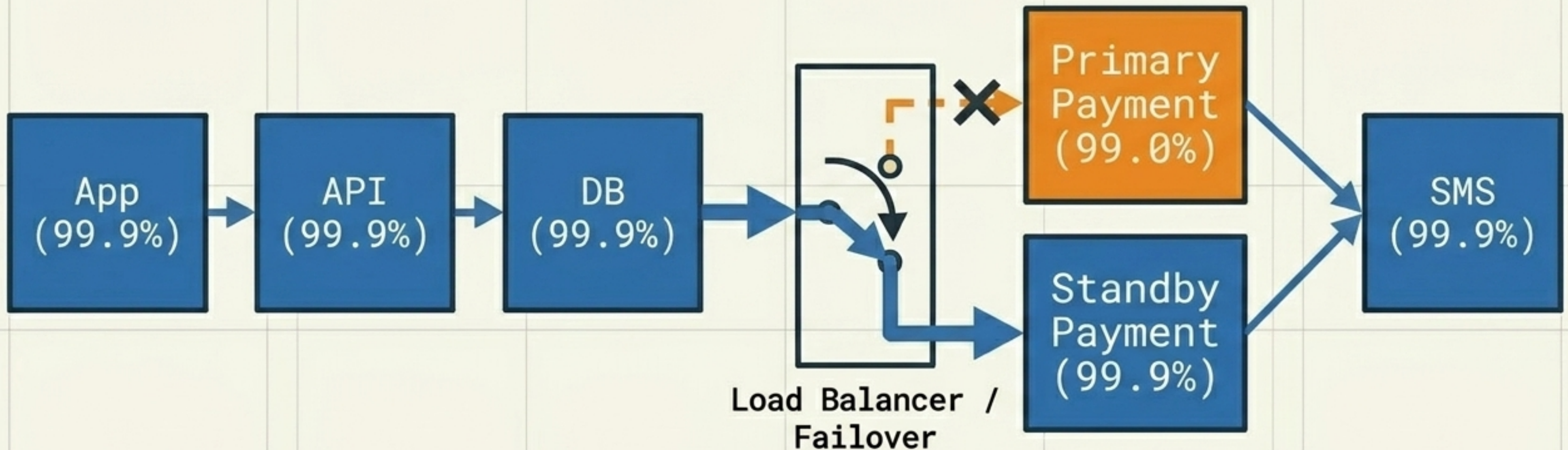
One weak link caps the entire system's uptime.



$$0.999 \times 0.999 \times 0.999 \times 0.990 \times 0.999 = 98.7\% \text{ Effective Availability}$$

The Redundancy Solution (N+1)

Spare capacity + Health checks = Automatic Failover



Do not wait for Twitter to tell you you are down

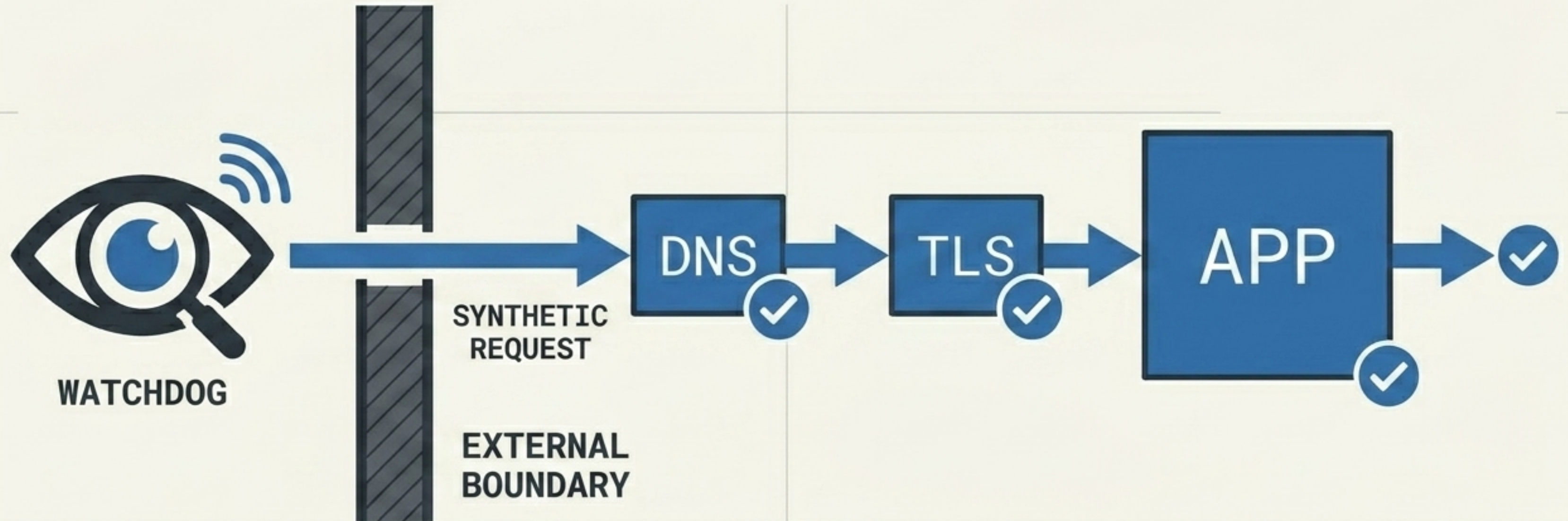
The Meta-Problem: Blind spots become incidents.

You need redundant monitoring to solve the SRE meta-availability problem.



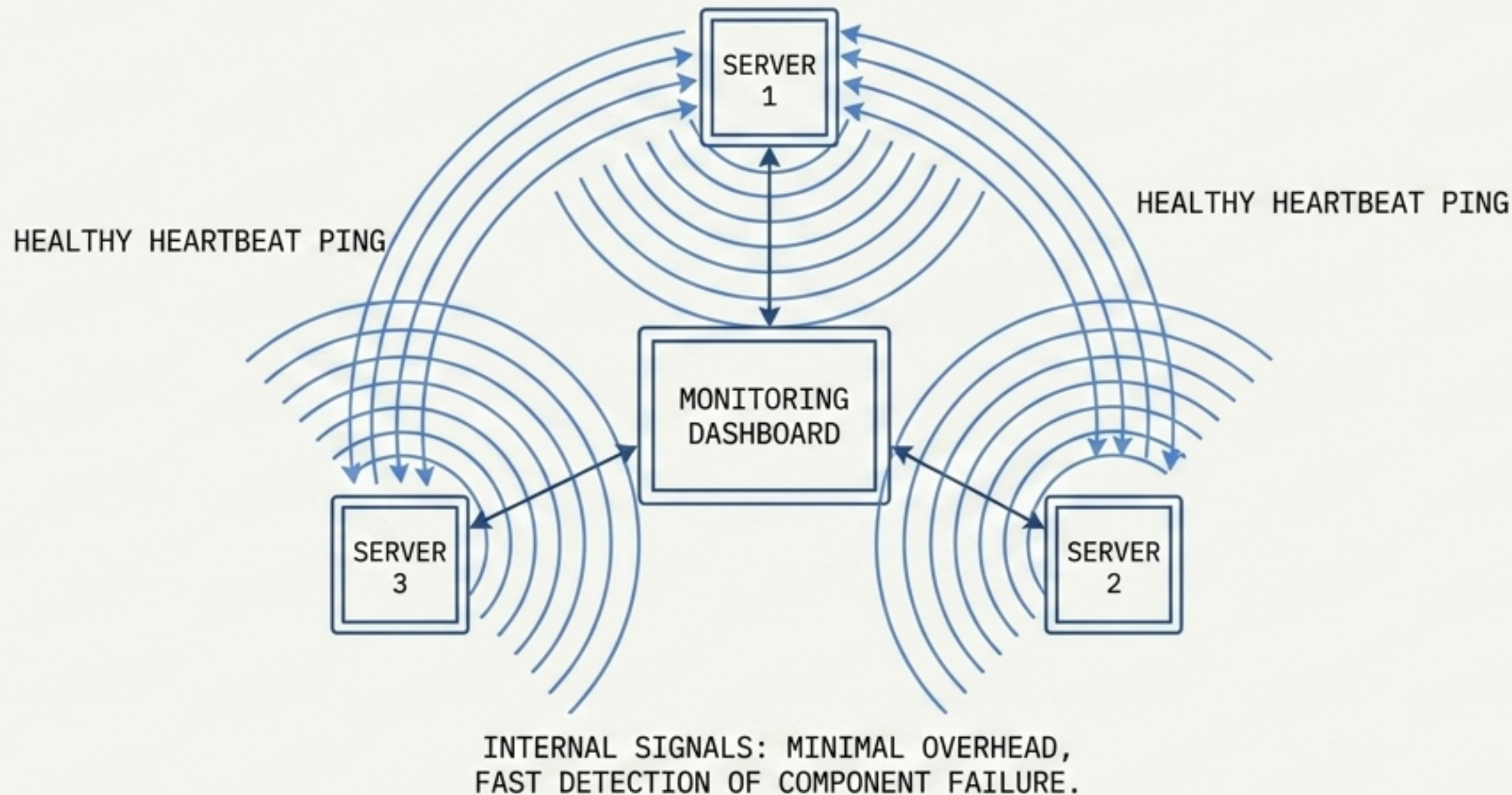
Active Monitors: The Watchdog

Synthetic requests from outside the network.
Closer to the true user perspective.



Passive Monitors: The Heartbeat

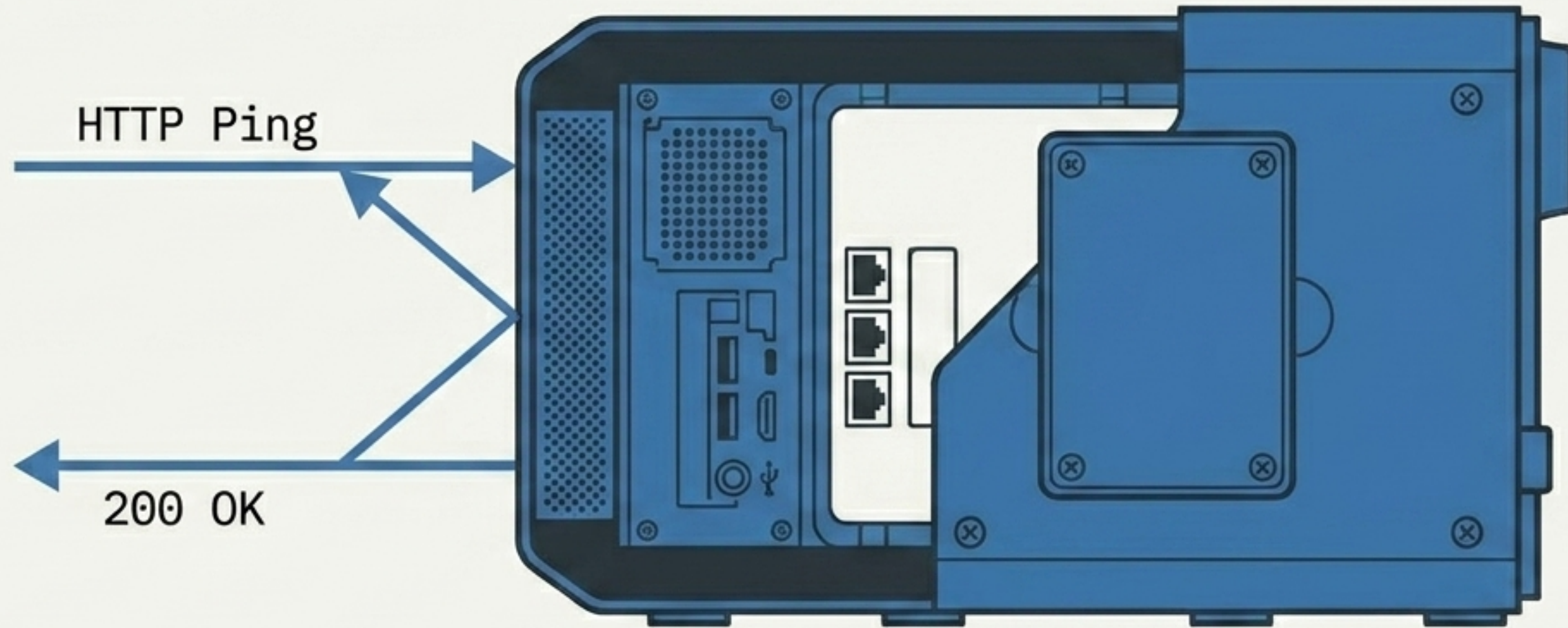
Periodic pings emitted from the inside.
A cheap liveness signal; silence implies failure.



The Illusion of Shallow Health

Liveness

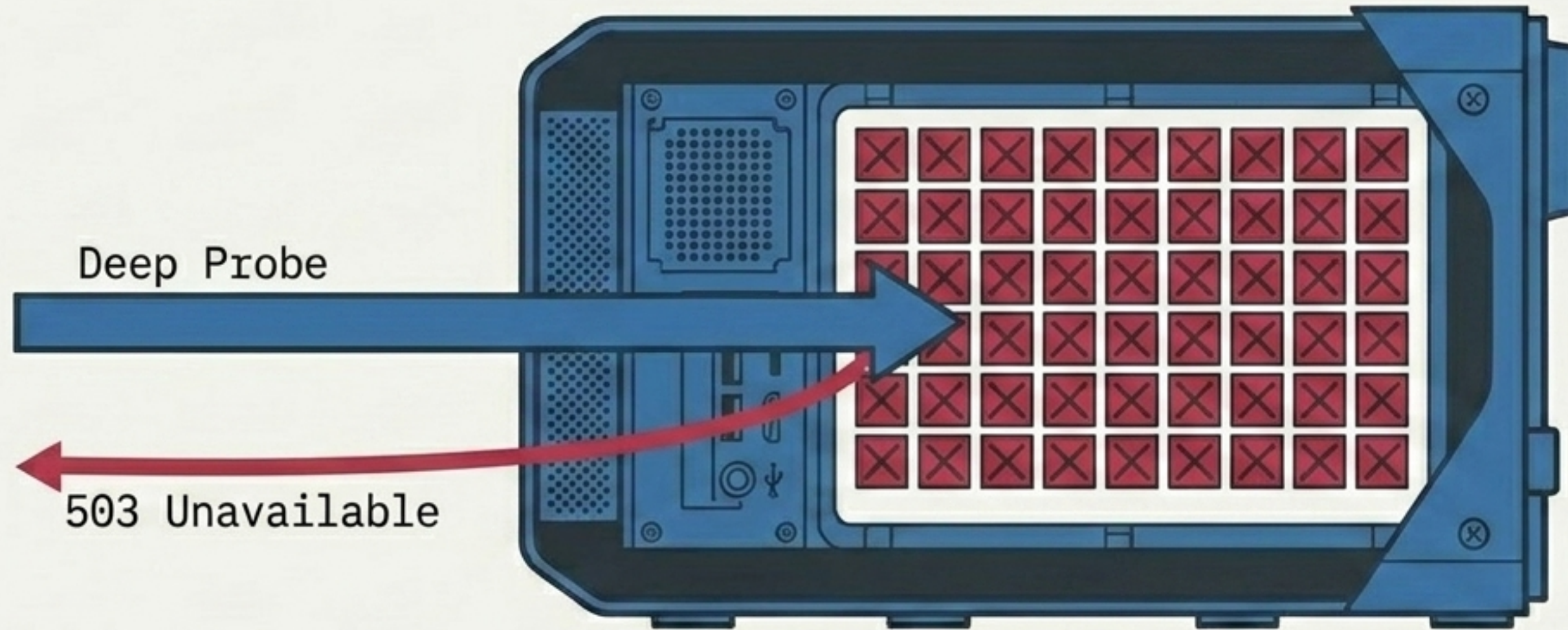
The endpoint returns 200 OK simply because the process is turned on.



The Reality of Deep Readiness

Readiness

Validates the actual dependencies needed to serve real traffic.



Overload is an Availability Incident

Saturation causes errors even if no code deployment occurred.

